

Activity 12 - Integrate components into a tested command-line tool

AI Vibe Coding with Rust / TGS-2023039924 / version 3.1 / K4/A4

Specification

`run(csv, threshold) -> Result<String, String>` returns "ITEMS \n\nTOTAL \$D.CC\nLOW STOCK a, b | none". The binary takes `<file> [threshold=5]`; exit 0 on success, 1 for unreadable or invalid data, 2 for usage errors.

Part 1: Learn Rust prompts

Explain it simply

I am learning Rust. Explain reading files, command-line arguments, stdout versus stderr and process exit codes to a beginner in plain English. Use a short example from a small shop's stock system, keep the code under 20 lines, show its output, and point out one mistake beginners often make.

Walk me through the sample

Here is `samples/01_read_file_and_exit_code.rs` from my course. Walk through it line by line. Before I run it, ask me to predict the output. Then suggest one small change I can make to experiment, and tell me what should happen.

Quiz me

Quiz me on reading files, command-line arguments, stdout versus stderr and process exit codes with 5 questions, one at a time. Wait for my answer each time and explain why it is right or wrong. Include one question where I predict the output or the compiler error of a short Rust snippet.

Compare with what I know

I know some Python or JavaScript. In a small table, compare how Rust handles building command-line tools with Python and JavaScript. Then show one Rust example and explain what the Rust compiler checks for me that the others do not.

Part 2: Build with AI (vibe-coding prompts)

Plan before code

Act as a senior Rust reviewer. Specification: `run(csv, threshold) -> Result<String, String>` returns "ITEMS \n\nTOTAL \$D.CC\nLOW STOCK a, b | none". The binary takes `<file> [threshold=5]`; exit 0 on success, 1 for unreadable or invalid data, 2 for usage errors. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

Implement "Integrate components into a tested command-line tool" in `starter/src` so that the supplied tests pass. Keep `main` thin; put every rule in `lib.rs` where the library tests can reach it. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

Review checklist

Signatures unchanged; no unsafe; no unwrap() on input; no new crates; every boundary tested; cargo test, clippy -D warnings and fmt --check pass; you can explain every line.

Part 3: Rust sample scripts (samples/ folder)

01_read_file_and_exit_code.rs

Run: `rustc --edition 2021 01_read_file_and_exit_code.rs && ./01_read_file_and_exit_code`
`../data/stock.csv`

// Sample: read a file named on the command line; exit code 1 if it fails.
use std::process::ExitCode;

```
fn main() -> ExitCode {
    let path = std::env::args()
        .nth(1)
        .unwrap_or_else(|| "missing.csv".to_string());
    match std::fs::read_to_string(&path) {
        Ok(text) => {
            println!("{path}: {} line(s)", text.lines().count());
            ExitCode::SUCCESS
        }
        Err(error) => {
            eprintln!("error: cannot read {path}: {error}");
            ExitCode::from(1)
        }
    }
}
```

Expected output:

`../data/stock.csv: 5 line(s)`

02_stdout_stderr.rs

Run: `rustc --edition 2021 02_stdout_stderr.rs && ./02_stdout_stderr`

// Sample: results to stdout, diagnostics to stderr, and an explicit exit code.

```
fn main() {
    println!("ITEMS 4"); // stdout: the report, safe to pipe into a file
    eprintln!("note: diagnostics go to stderr"); // stderr: messages for people
    let code = if std::env::args().count() > 1 { 2 } else { 0 };
    println!("exit code will be {code}");
    std::process::exit(code);
}
```

Expected output:

`ITEMS 4`
`exit code will be 0`

Learn more

Rust Book: An I/O Project: Building a Command Line Program:

<https://doc.rust-lang.org/book/ch12-00-an-io-project.html>

Rust official learning hub (the Book, Rustlings, Rust by Example): <https://rust-lang.org/learn/>

W3Schools Rust tutorial: <https://www.w3schools.com/rust/>

W3Schools: Get Started with Rust: https://www.w3schools.com/rust/rust_getstarted.php